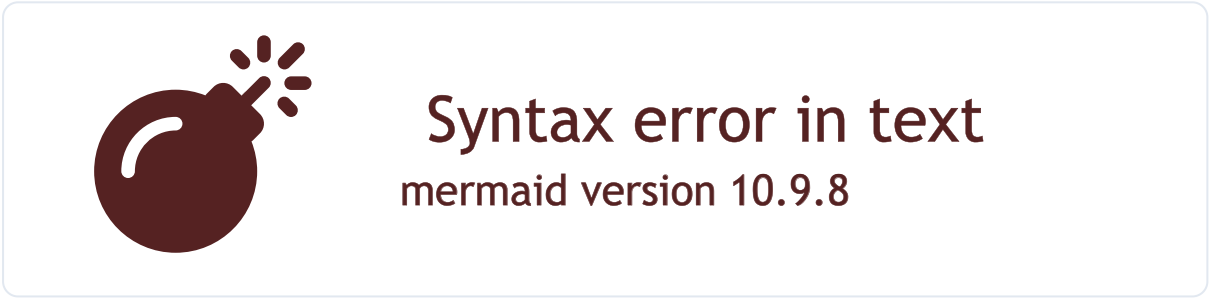
Lecture 21.14: Determinants

📌 Topic Overview

In this lecture, Shraddha ma'am introduces **Determinants**, establishing that a determinant $|A|$ is a **single scalar number calculated from a square matrix**. She derives the 2×2 formula $|A| = ad - bc$ on the blackboard, covers 3×3 cofactor expansion, and explains its geometric interpretation as the area/volume scaling factor of linear transformations.

🏛️ Geometric Scaling & Invertibility



📐 Mathematical Formulation

1. 2×2 Matrix Determinant:

For $\mathbf{A} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$:

$$|\mathbf{A}| = \det(\mathbf{A}) = ad - bc$$

Example:

$$\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 1 & 3 \end{bmatrix} \implies |\mathbf{A}| = (4 \times 3) - (2 \times 1) = 12 - 2 = 10$$

2. 3×3 Matrix Determinant (Cofactor Expansion):

For $\mathbf{A} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$:

$$|\mathbf{A}| = a\begin{vmatrix} e & f \\ h & i \end{vmatrix} - b\begin{vmatrix} d & f \\ g & i \end{vmatrix} + c\begin{vmatrix} d & e \\ g & h \end{vmatrix}$$

$$|\mathbf{A}| = a(ei - fh) - b(di - fg) + c(dh - eg)$$

3. Machine Learning Connection: Invertibility

In Ordinary Least Squares (OLS) Linear Regression, the analytical weight vector is solved as:

$$\mathbf{w}^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

If $\det(\mathbf{X}^T\mathbf{X}) = 0$ (due to multicollinear redundant features), the matrix cannot be inverted!

💻 Python Implementation

```
import numpy as np

def compute_determinant_with_steps(matrix_2x2=None, matrix_3x3=None):
    """
    Computes determinants of 2x2 and 3x3 square matrices.
    2x2 formula: det(A) = a*d - b*c
    3x3 cofactor expansion across the first row.
    Explains geometric area/volume scaling and matrix invertibility.

    Returns:
        dict: Determinant values and invertibility flags.
    """
    if matrix_2x2 is None:
        # Shraddha ma'am's blackboard formula: A = [[a, b], [c, d]] -> ad - bc
        matrix_2x2 = np.array([
            [4, 2],
            [1, 3]
        ])
    if matrix_3x3 is None:
        matrix_3x3 = np.array([
            [1, 2, 3],
            [0, 1, 4],
            [5, 6, 0]
        ])

    # 2x2 Determinant
    a, b = matrix_2x2[0, 0], matrix_2x2[0, 1]
    c, d = matrix_2x2[1, 0], matrix_2x2[1, 1]
    det_2x2_formula = float(a * d - b * c)
    det_2x2_numpy = float(np.linalg.det(matrix_2x2))

    # 3x3 Determinant via NumPy
    det_3x3_numpy = float(np.linalg.det(matrix_3x3))

    return {
        "matrix_2x2": matrix_2x2.tolist(),
        "det_2x2_formula": det_2x2_formula,
        "det_2x2_numpy": det_2x2_numpy,
        "is_2x2_invertible": bool(not np.isclose(det_2x2_numpy, 0.0)),
        "matrix_3x3": matrix_3x3.tolist(),
        "det_3x3_numpy": det_3x3_numpy,
        "is_3x3_invertible": bool(not np.isclose(det_3x3_numpy, 0.0))
    }

# --- Execution & Verification ---
res = compute_determinant_with_steps()
print("2x2 Matrix:")
print(np.array(res["matrix_2x2"]))
print(f"det(A) = ad - bc = 4*3 - 2*1 = 12 - 2 = {res['det_2x2_formula']:.1f}")
```

```
print("Is 2x2 Invertible (det != 0)?", res["is_2x2_invertible"])
print(f"3x3 Matrix Determinant: {res['det_3x3_numpy']:.2f}")
```

🔑 Key Takeaways

1. Determinants are defined strictly for **square matrices** ($n \times n$).
2. The determinant is a single scalar representing the geometric scaling factor of area or volume.
3. If $\det(A) = 0$, the matrix is singular and cannot be inverted.